

[Skip to content](#)

Chat history

You said:

diehard test for randomness test list

ChatGPT said:

Here's the **classic Diehard** battery (Marsaglia) — i.e., the usual “Diehard tests” people refer to when they say *Diehard randomness test list*:

1. **Birthday Spacings**
2. **Overlapping 5-Permutations**
3. **Binary Rank Test (31×31 matrices)**
4. **Binary Rank Test (32×32 matrices)**
5. **Binary Rank Test (6×8 matrices)**
6. **Bitstream Test**
7. **Overlapping-Pairs-Sparse-Occupancy (OPSO)**
8. **Overlapping-Quadruples-Sparse-Occupancy (OQSO)**
9. **DNA Test**
10. **Count-the-1's (stream of bytes)**
11. **Count-the-1's (specific bytes)**
12. **Parking Lot Test**
13. **Minimum Distance Test**
14. **3D Spheres Test**
15. **Squeeze Test**
16. **Overlapping Sums Test**
17. **Runs Test**
18. **Craps Test**

If you want, tell me **what format your data is** (bytes? 32-bit ints? file size), and I'll tell you which Diehard tests are most appropriate and how to feed the file (e.g., for dieharder / testu01).

You said:

i have read comt the no of test both bytes and stream then bday test then parking test now which test i shol d read

ChatGPT said:

If you've already done **Count-the-1's (byte + stream)**, **Birthday Spacings**, and **Parking Lot**, the next ones I'd read/do (in a good, practical order) are:

1) Binary Rank tests (matrix rank)

Do these next because they're **core linear-dependence detectors** and easy to interpret.

- **Rank 31×31**
- **Rank 32×32**
- **Rank 6×8**

What they catch: linear artifacts (common in LFSR-like or poorly mixed generators).

2) Runs test

Very standard and intuitive.

- **Runs**

What it catches: too many/too few alternating streaks → correlation in bits.

3) Squeeze test

Good “nonlinear” stress test; not too hard conceptually.

- **Squeeze**

What it catches: subtle non-uniformity via repeated modular reductions.

4) Minimum Distance and 3D Spheres

These are the next “geometry/spacing” family after Parking Lot.

- **Minimum Distance**
- **3D Spheres**

What they catch: clustering / lattice structure in points derived from the stream.

5) OPSO, OQSO, DNA

These are “sparse occupancy / word” structure tests; very useful.

- **OPSO**
- **OQSO**
- **DNA**

What they catch: patterns in overlapping bit-words (including weaknesses that don't show in simpler tests).

6) Overlapping Sums and Craps (finishers)

- **Overlapping Sums**
- **Craps**

What they catch: distributional issues under specific transforms; Craps is more “fun” but still informative.

The exact recommended next test depends on your generator type

- If it's **crypto / ChaCha / AES / hash-based**: do **Binary Rank** → **OPSO/OQSO/DNA** → **Minimum Distance/3D Spheres**.
 - If it's **LCG / xorshift / LFSR / custom PRNG**: do **Binary Rank immediately**, then **Runs**, then **Bitstream/OPSO/OQSO**.
-

Minimal “next 5” you should do now

1. Rank 31×31
2. Rank 32×32
3. Rank 6×8
4. Runs
5. OPSO (or OQSO)

If you tell me how your data is stored (e.g., “I have 1024×500×4 bytes” or “32-bit integers”), I'll map **exactly** how many bits/words each of these consumes and what output (p-values / chi-square / df) you should expect to compute.

You said:

im making ny die hard suit so which tst shold ne nxt

ChatGPT said:

If you're **building your own Diehard suite**, the next test you should implement is:

Next: Binary Rank Test (start with 32×32)

It's one of the most diagnostic Diehard tests, and it's straightforward to code once you have bit-unpacking.

Implement in this order

1. **Binary Rank 32×32**
2. **Binary Rank 31×31**
3. **Binary Rank 6×8**

Why this is the right “next”

- It detects **linear dependence** (a very common PRNG weakness).
- Clean pipeline: read bits → build matrix → compute rank over GF(2) → compare to expected rank distribution → chi-square → p-value.

Output you should produce (Diehard style)

- Count how many matrices have rank:
 - **32** (full rank)
 - **31**
 - **≤ 30**
 - Then do a **chi-square goodness-of-fit** vs the theoretical probabilities for these three bins, and report the **p-value**.
-

After Binary Rank, implement: Runs Test

Because it’s the easiest correlation test to add after you already handle bits.

Practical “build order” for your suite (recommended)

1. Count-the-1’s (you already did)
2. Birthday spacings (you did)
3. Parking lot (you did)
4. **Binary Rank (next)**
5. **Runs**
6. Minimum Distance
7. 3D Spheres
8. OPSO / OQSO / DNA
9. Squeeze
10. Overlapping Sums
11. Craps

If you want, I can give you **exact pseudocode + formulas** for the **32×32 rank test** (GF(2) elimination, expected probabilities, chi-square df, p-value), in the same “Diehard report” format.

You said:

Binary Rank 32×32 explain this test like i dont know anything step to step

ChatGPT said:

Sure. Think of **Binary Rank 32×32** as: *“Take chunks of bits, arrange them into a 32×32 grid of 0/1, and see how often that grid has full ‘information’ (full rank). Bad RNGs produce too many dependent rows/columns.”*

What “rank” means here (in plain terms)

- You make a **32×32 matrix** of bits (0/1).
- Each row is a 32-bit pattern.
- **Rank = 32** means all 32 rows contain **independent** information (none can be built by XOR-combining the others).
- **Rank < 32** means some rows are redundant (linear patterns).

All arithmetic is over **GF(2)**:

- add = XOR
 - subtract = XOR
 - multiply = AND
-

Step-by-step procedure

Step 1: Read bits from your random stream

You need **1024 bits** per matrix because:

- $32 \times 32 = 1024$ entries.

So you do:

- Read the next 1024 bits from the file/stream.

Step 2: Fill a 32×32 matrix with those bits

Common convention:

- Row 0 gets the first 32 bits,
- Row 1 gets the next 32 bits,
- ...
- Row 31 gets the next 32 bits.

So each row is basically one **32-bit word**.

Step 3: Compute the matrix rank over GF(2)

You do Gaussian elimination, but with XOR operations.

Algorithm idea (row-reduction using XOR):

1. Set rank = 0.

2. For each column $c = 0..31$:

- Find a row $r \geq \text{rank}$ that has a **1** in column c .
- If you find it:
 - Swap that row with row rank .
 - For every other row $i \neq \text{rank}$ that also has a 1 in column c :
 - $\text{Row}[i] = \text{Row}[i] \text{ XOR } \text{Row}[\text{rank}]$
 - Increase $\text{rank}++$.
- If you don't find any 1 in that column, move to next column.

At the end, rank is $0..32$.

Implementation tip: Store each row as a 32-bit integer. Then XOR is fast.

Step 4: Repeat many times

You repeat Steps 1–3 for **M matrices** (M should be large, like thousands or more).
For each matrix, you record its rank.

You keep three counts:

- N_{32} = number of matrices with rank 32
- N_{31} = number with rank 31
- N_{30} = number with rank ≤ 30 (Diehard groups all ≤ 30 together)

Check: $N_{32} + N_{31} + N_{30} = M$.

Step 5: Compare with what a truly random generator should produce

For a perfectly random bit matrix, the rank is **usually** 32, sometimes 31, rarely ≤ 30 .

For 32×32 , the theoretical probabilities are approximately:

- $P(\text{rank} = 32) = 0.288788095$
- $P(\text{rank} = 31) = 0.577576190$
- $P(\text{rank} \leq 30) = 0.133635715$

(These are standard Diehard constants for this test.)

So the expected counts are:

- $E_{32} = M * P_{32}$
 - $E_{31} = M * P_{31}$
 - $E_{30} = M * P_{30}$
-

Step 6: Compute a chi-square goodness-of-fit statistic

$$\chi^2 = \frac{(N_{32}-E_{32})^2}{E_{32}} + \frac{(N_{31}-E_{31})^2}{E_{31}} + \frac{(N_{30}-E_{30})^2}{E_{30}}.$$

Degrees of freedom

You have 3 bins, and no parameters estimated from data, so:

$$df = 3 - 1 = 2.$$

Step 7: Convert chi-square to a p-value

Compute:

$$p = P(\chi_{df=2}^2 \geq \chi_{\text{obs}}^2).$$

Interpretation:

- p very close to 0 or 1 repeatedly → suspicious (non-random).
- p values should look “uniform” over [0,1] across repeated runs/windows.

“Sanity checks” while implementing

- Make sure each bin has **expected count** ≥ 5 (the classic rule for chi-square validity). This means choose M large enough so $E_{30} = 0.1336 \cdot M \geq 5 \Rightarrow \mathbf{M \geq 38}$ (but you want way bigger, like 1000+).
- If your RNG is good, you’ll often see N31 about twice N32.

Minimal pseudo-code (logic only)

1. M = number_of_matrices
2. loop j=1..M:
 - read 32 uint32 words (or 1024 bits)
 - treat them as 32 rows
 - rank = gf2_rank_32(rows)
 - if rank==32: N32++
else if rank==31: N31++
else: N30++
3. compute E32,E31,E30
4. compute chi-square, df=2, p-value

5. report

If you tell me **your stream format** (do you already have 32-bit integers? or raw bytes?), I'll give you an exact implementation recipe for reading bits/words correctly and a clean GF(2) rank routine (C/C++).

You said:

how much data needed all things tell about thi test i dont now anything

ChatGPT said:

Binary Rank 32×32: how much data you need (and why)

This test consumes your random stream in **blocks of 1024 bits** at a time:

- A **32×32** matrix has **32×32 = 1024 bits**
- **1024 bits = 128 bytes**
- Equivalently: **32 words of 32-bit** (because 32×32 = 1024 bits)

So if you test **M matrices**, the raw data required is:

$$\text{Data} = 1024M \text{ bits} = 128M \text{ bytes.}$$

Concrete sizes

Matrices (M)	Bytes needed	Approx
100	12,800	12.5 KB
1,000	128,000	125 KB
10,000	1,280,000	1.22 MB
100,000	12,800,000	12.2 MB
1,000,000	128,000,000	122 MB

Minimum M you *can* use (chi-square validity)

At the end you put results into **3 bins**:

- rank = 32

- rank = 31
- rank $\leq$ 30

Expected probabilities (for truly random bits) are approximately:

- $P_{32} = 0.288788095$
- $P_{31} = 0.577576190$
- $P_{\leq 30} = 0.133635715$

For a chi-square test, each expected count should be ≥ 5 , especially the smallest bin $P_{\leq 30}$:

$$E_{\leq 30} = M \cdot 0.1336 \geq 5 \Rightarrow M \geq 37.4$$

So **absolute minimum** is about:

- **M = 40 matrices** $\rightarrow$ data $\approx 40 \times 128 = 5120$ bytes (~5 KB)

But that's *way too small* to learn anything reliably.

Recommended M (what you should actually use)

If you're making a serious Diehard-style suite:

- **M = 10,000 matrices** (good baseline)
 $\rightarrow$ data = **1.28 MB**
- **M = 100,000 matrices** (stronger)
 $\rightarrow$ data = **12.8 MB**

These sizes make the observed frequencies stable and the p-value meaningful.

If you want multiple p-values ("windows")

A common practice is: don't just compute **one** p-value; compute many p-values from independent chunks.

Example:

- Choose **M = 10,000 matrices per run** (1.28 MB per run)
- Do **R = 20 runs** $\rightarrow$ total data $\approx$ **25.6 MB**
- You'll get 20 p-values; they should look roughly uniform on [0,1].

This is much more informative than one huge run, because a broken generator often fails intermittently or in specific bit regions.

What exactly you report (Diehard style)

For each run:

1. Read M matrices (each 128 bytes).
2. Count:
 - $N_{32}, N_{31}, N_{\leq 30}$
3. Compute expected counts:
 - $E_{32} = MP_{32}$, etc.
4. Compute chi-square:

$$\chi^2 = \sum_k \frac{(N_k - E_k)^2}{E_k}$$

5. Degrees of freedom: **df = 2** (3 bins $\rightarrow$ 3-1)
6. Compute **p-value** from χ^2 with df=2.

One practical warning (important)

Some generators have “good high bits, bad low bits” (or the reverse).
So for your own suite, it’s smart to test:

- raw 32-bit words as-is, **and/or**
- also test after bit-reversal or using only LSB/MSB streams (optional but useful)

If you tell me your file format (raw bytes? 32-bit ints? little-endian?), I’ll give you an exact “read loop” (how to slice the stream into 32×32 matrices) and a clean GF(2) rank routine you can drop into C/C++.

You said:

in original diehard test how much data needed for this test

ChatGPT said:

Thought for 7s

In the **original Marsaglia Diehard** battery, the **32×32 Binary Rank** test uses:

- **40,000 matrices** (this is fixed in the classic Diehard implementation).
- Each matrix needs **32 random 32-bit integers** (one integer per row).

So how much data is that?

Per matrix:

- 32 integers $\times$ 4 bytes/integer = **128 bytes** (= 1024 bits)

Total:

- $40,000 \times 128 \text{ bytes} = \mathbf{5,120,000 \text{ bytes}}$

Let me multiply explicitly to avoid mistakes:

- $128 \times 40,000$
 $= (128 \times 4) \times 10,000$
 $= 512 \times 10,000$
 $= \mathbf{5,120,000 \text{ bytes} \approx 5.12 \text{ MB}} (\approx 4.88 \text{ MiB})$

So, **original Diehard needs about ~5.1 MB of data** for the **Binary Rank 32×32** test.

You said:

sonow tell stepmto step how this test works in original diehard version step to step like i domt knw anything

ChatGPT said:

Here is **exactly how the Binary Rank 32×32 test works in the original Diehard**, explained from zero.

What the test is trying to catch

A “bad” RNG often produces bits with **linear patterns** (some bits are XOR-combinations of others). This test detects that by checking how often a random **32×32 bit matrix** has **full rank**.

What data Diehard uses (original)

Diehard takes:

- **40,000 matrices**
- Each matrix uses **32 random 32-bit integers**
- So per matrix = $32 \times 32 \text{ bits} = \mathbf{1024 \text{ bits} = 128 \text{ bytes}}$
- Total data = $40,000 \times 128 \text{ bytes} = \mathbf{5,120,000 \text{ bytes} (\sim 5.12 \text{ MB})}$

Step-by-step: what Diehard does

Step 1: Read 32-bit numbers from your stream

Diehard reads the file as a sequence of **32-bit unsigned integers**.

For each matrix it needs **32 integers**:

$$x_1, x_2, \dots, x_{32}$$

Step 2: Convert those 32 integers into a 32×32 matrix of bits

Each 32-bit integer becomes **one row** of the matrix.

- Row 1 = bits of x_1
- Row 2 = bits of x_2
- ...
- Row 32 = bits of x_{32}

So you now have a matrix A with entries 0/1:

$$A \in \{0,1\}^{32 \times 32}$$

Important: all arithmetic is in **GF(2)** (binary math):

- addition = XOR
- subtraction = XOR (same as addition)
- multiplication = AND

Step 3: Compute the rank of this matrix over GF(2)

Now Diehard computes:

$$r = \text{rank}_{\text{GF}(2)}(A)$$

So r can be 0,1,2,...,32.

How rank is computed (conceptually):

Diehard performs **Gaussian elimination**, but using XOR.

- Find a row with a 1 in the current pivot column
- Swap it into pivot position
- XOR it into other rows to clear that column
- Move to next pivot

If you can find 32 pivots $\rightarrow$ rank 32 (full rank).

If only 31 pivots $\rightarrow$ rank 31.

If fewer $\rightarrow$ rank ≤ 30 .

Step 4: Repeat for 40,000 matrices and count outcomes

Diehard repeats Steps 1–3 **40,000 times** and counts:

- N_{32} : number of matrices with rank 32
- N_{31} : number of matrices with rank 31
- $N_{\leq 30}$: number with rank 30 or less

Check:

$$N_{32} + N_{31} + N_{\leq 30} = 40000$$

Step 5: Compare your counts to what “true randomness” predicts

For a truly random 32×32 bit matrix, the rank distribution is known (constants used by Diehard):

- $P_{32} \approx 0.288788095$
- $P_{31} \approx 0.577576190$
- $P_{\leq 30} \approx 0.133635715$

So Diehard computes expected counts:

$$E_{32} = 40000 P_{32}, E_{31} = 40000 P_{31}, E_{\leq 30} = 40000 P_{\leq 30}$$

Numerically (approx):

- $E_{32} \approx 11551.5$
- $E_{31} \approx 23103.0$
- $E_{\leq 30} \approx 5345.4$

Step 6: Compute a chi-square statistic (goodness-of-fit)

Diehard uses:

$$\chi^2 = \frac{(N_{32} - E_{32})^2}{E_{32}} + \frac{(N_{31} - E_{31})^2}{E_{31}} + \frac{(N_{\leq 30} - E_{\leq 30})^2}{E_{\leq 30}}$$

Degrees of freedom:

- 3 bins $\rightarrow df = 3 - 1 = 2$

Step 7: Convert chi-square to a p-value

Diehard computes:

$$p = P(\chi_{df=2}^2 \geq \chi_{\text{obs}}^2)$$

This **p-value** is what Diehard prints for this test.

How to interpret the p-value

- A single p-value near 0.5 is “typical”.

- A p-value extremely close to 0 (like < 0.000001) or extremely close to 1 repeatedly is suspicious.
- In a full suite, you look for **systematic extremes** across tests, not one random low p-value.

The whole test in one sentence

Take 40,000 blocks of 1024 bits, form 32×32 bit matrices, compute their GF(2) ranks, and check whether the counts of rank 32 / 31 / ≤30 match the known random distribution using a chi-square p-value.

If you want, I can also give you:

- the **exact GF(2) rank algorithm using 32-bit rows (fast XOR elimination)**, and
- the exact output format (like Diehard prints) so your suite matches the original closely.

You said:

can you write the code of this in .cpp and i have dat file of 5,120,000 bytes where values are in hex and write a simple code easy to explainable with good speed dont take much time

ChatGPT said:

```
// diehard_rank32.cpp // // Binary Rank 32x32 test (original Diehard style): // - Consumes 40,000
matrices // - Each matrix = 32 rows x 32 bits = 1024 bits = 128 bytes // - Input file here is TEXT
containing HEX BYTES (e.g. "0A FF 1c ..."). // // Output: // - N32, N31, NLE30 // - chi-square (df=2)
and p-value (for df=2, p = exp(-chi2/2)) // // Compile (g++): // g++ -O3 -std=c++17
diehard_rank32.cpp -o diehard_rank32 // // Run: // ./diehard_rank32 yourfile.dat // // If your 32-bit
words are stored little-endian vs big-endian in the hex-byte stream, // change
WORD_LITTLE_ENDIAN below. #include <cstdlib> #include <cstdlib> #include <cmath> #include
<iostream> #include <fstream> #include <string> #include <vector> static constexpr bool
WORD_LITTLE_ENDIAN = true; // set false if your word order is big-endian // Diehard probabilities
for rank(32x32) over GF(2) static constexpr double P32 = 0.288788095; static constexpr double P31 =
0.577576190; static constexpr double PLE30 = 0.133635715; // Read next hex byte token from
stream. // Accepts tokens like: "0A", "0a", "0A," (comma/semicolon stripped), "0x0A" also ok. static
bool read_hex_byte(std::istream& in, uint8_t& outByte) { std::string tok; while (in >> tok) { // Strip
common separators at end while (!tok.empty() && (tok.back() == ',' || tok.back() == ';'))
tok.pop_back(); if (tok.empty()) continue; // Allow optional leading 0x / 0X if (tok.size() >= 2 &&
tok[0] == '0' && (tok[1] == 'x' || tok[1] == 'X')) { tok = tok.substr(2); } if (tok.empty()) continue; //
Parse hex char* endp = nullptr; long v = std::strtoul(tok.c_str(), &endp, 16); if (endp == tok.c_str() ||
*endp != '\0' || v < 0 || v > 255) { // Not a valid hex byte token; skip it (robust to stray text) continue;
} outByte = static_cast<uint8_t>(v); return true; } return false; // EOF } // Compute rank of 32x32
binary matrix over GF(2), rows stored as uint32 (each row = 32 bits). static int gf2_rank_32(uint32_t
rows[32]) { int rank = 0; // We pivot from MSB to LSB (bit 31 down to 0) for consistency. for (int col =
31; col >= 0 && rank < 32; --col) { const uint32_t mask = (1u << col); // Find a pivot row with a 1 in
this column at/below 'rank' int pivot = -1; for (int r = rank; r < 32; ++r) { if (rows[r] & mask) { pivot = r;
break; } } if (pivot < 0) continue; // no pivot in this column // Swap pivot row into position 'rank' if
(pivot != rank) { uint32_t tmp = rows[pivot]; rows[pivot] = rows[rank]; rows[rank] = tmp; } //
```

```

Eliminate this bit from all other rows (XOR), to get row-reduced form for (int r = 0; r < 32; ++r) { if (r !=
rank && (rows[r] & mask)) { rows[r] ^= rows[rank]; } } ++rank; } return rank; } static uint32_t
bytes_to_u32(const uint8_t b[4]) { if (WORD_LITTLE_ENDIAN) { return (uint32_t)b[0] |
((uint32_t)b[1] << 8) | ((uint32_t)b[2] << 16) | ((uint32_t)b[3] << 24); } else { return ((uint32_t)b[0]
<< 24) | ((uint32_t)b[1] << 16) | ((uint32_t)b[2] << 8) | (uint32_t)b[3]; } } int main(int argc, char**
argv) { if (argc < 2) { std::cerr << "Usage: ./diehard_rank32 <hex_byte_text_file>\n"; return 1; }
std::ifstream fin(argv[1]); if (!fin) { std::cerr << "Error: cannot open file: " << argv[1] << "\n"; return 1;
} // Original Diehard uses 40,000 matrices for Rank 32x32 const int M_target = 40000; int64_t N32 =
0, N31 = 0, NLE30 = 0; int matrices_done = 0; // For each matrix: read 128 bytes => 32 uint32 rows
for (; matrices_done < M_target; ++matrices_done) { uint32_t rows[32]; for (int r = 0; r < 32; ++r) {
uint8_t b[4]; for (int k = 0; k < 4; ++k) { if (!read_hex_byte(fin, b[k])) { // Not enough data
matrices_done = r == 0 ? matrices_done : matrices_done + 1; // keep consistent std::cerr << "EOF:
not enough hex bytes to complete matrix " << (matrices_done+1) << "\n"; // Break out cleanly goto
done_reading; } } rows[r] = bytes_to_u32(b); } const int rank = gf2_rank_32(rows); if (rank == 32)
++N32; else if (rank == 31) ++N31; else ++NLE30; } done_reading: const int64_t M = N32 + N31 +
NLE30; if (M == 0) { std::cerr << "No matrices processed.\n"; return 1; } // Expected counts const
double E32 = (double)M * P32; const double E31 = (double)M * P31; const double ELE30 =
(double)M * PLE30; // Chi-square statistic (3 bins -> df=2) const double chi2 = ( (N32 - E32) * (N32 -
E32) ) / E32 + ( (N31 - E31) * (N31 - E31) ) / E31 + ( (NLE30 - ELE30) * (NLE30 - ELE30) ) / ELE30; // For
df=2, survival function is p = exp(-chi2/2) const double p_value = std::exp(-0.5 * chi2); std::cout <<
"Binary Rank 32x32 (Diehard-style)\n"; std::cout << "Processed matrices M = " << M << "\n";
std::cout << "Counts:\n"; std::cout << " N(rank=32) = " << N32 << "\n"; std::cout << " N(rank=31) = "
<< N31 << "\n"; std::cout << " N(rank<=30) = " << NLE30 << "\n"; std::cout << "Expected (from
Diehard probabilities):\n"; std::cout << " E32 = " << E32 << "\n"; std::cout << " E31 = " << E31 << "\n";
std::cout << " E<=30 = " << ELE30 << "\n"; std::cout << "chi^2 (df=2) = " << chi2 << "\n"; std::cout <<
"p-value = " << p_value << "\n"; // Simple interpretation hint if (p_value < 1e-6 || p_value > 1.0 - 1e-
6) { std::cout << "WARNING: Extreme p-value (very close to 0 or 1) is suspicious.\n"; } return 0; }

```

Notes (important, but short)

- Your file is **hex text**, so this reads tokens like AF, 0xAF, af, etc.
- The only “gotcha” is **endianness** of 32-bit words. If your hex stream is written as bytes in the same order Diehard expects on a typical PC, keep WORD_LITTLE_ENDIAN = true. If results look wildly off, flip it to false and re-run.

If you paste **the first ~32 bytes** of your hex file exactly as it appears, I can tell you which endianness is the right one.

You said:

use using namespace std and write also code for fetching file output.dat

ChatGPT said:

```

// diehard_rank32.cpp // // Binary Rank 32x32 test (original Diehard style). // INPUT : output.dat
(TEXT file containing HEX BYTES, e.g. "0A FF 1c ...") // OUTPUT : counts + chi-square + p-value // //
Data need (original Diehard): 40,000 matrices × 128 bytes = 5,120,000 bytes (as hex-byte tokens) //
// Compile: // g++ -O3 -std=c++17 diehard_rank32.cpp -o diehard_rank32 // // Run (reads output.dat
by default): // ./diehard_rank32 // // Or specify file: // ./diehard_rank32 output.dat // // If your 32-
bit word byte-order is different, flip WORD_LITTLE_ENDIAN. #include <cstdint> #include <cstdlib>

```

```

#include <cmath> #include <iostream> #include <fstream> #include <string> using namespace std;
static constexpr bool WORD_LITTLE_ENDIAN = true; // set false if your word order is big-endian //
Diehard theoretical probabilities (32x32 GF(2) rank) static constexpr double P32 = 0.288788095;
static constexpr double P31 = 0.577576190; static constexpr double PLE30 = 0.133635715; // Read
next hex byte token from stream (robust to separators, optional 0x prefix). static bool
read_hex_byte(istream& in, uint8_t& outByte) { string tok; while (in >> tok) { // strip trailing
separators like ", " "; " : " etc. while (!tok.empty() && (tok.back() == ',' || tok.back() == ';' || tok.back()
== ':')) tok.pop_back(); if (tok.empty()) continue; // optional leading 0x if (tok.size() >= 2 && tok[0] ==
'0' && (tok[1] == 'x' || tok[1] == 'X')) { tok = tok.substr(2); } if (tok.empty()) continue; char* endp =
nullptr; long v = strtoul(tok.c_str(), &endp, 16); if (endp == tok.c_str() || *endp != '\0' || v < 0 || v >
255) { // not a clean hex-byte token -> skip continue; } outByte = static_cast<uint8_t>(v); return true;
} return false; // EOF } static uint32_t bytes_to_u32(const uint8_t b[4]) { if (WORD_LITTLE_ENDIAN) {
return (uint32_t)b[0] | ((uint32_t)b[1] << 8) | ((uint32_t)b[2] << 16) | ((uint32_t)b[3] << 24); } else {
return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) | ((uint32_t)b[2] << 8) | (uint32_t)b[3]; } } //
Rank of 32x32 binary matrix over GF(2), rows as uint32. static int gf2_rank_32(uint32_t rows[32]) {
int rank = 0; // pivot columns from MSB->LSB (31..0) for (int col = 31; col >= 0 && rank < 32; --col) {
uint32_t mask = (1u << col); // find pivot row at/below 'rank' int pivot = -1; for (int r = rank; r < 32;
++r) { if (rows[r] & mask) { pivot = r; break; } } if (pivot < 0) continue; // swap pivot into position if
(pivot != rank) { uint32_t tmp = rows[pivot]; rows[pivot] = rows[rank]; rows[rank] = tmp; } //
eliminate from all other rows for (int r = 0; r < 32; ++r) { if (r != rank && (rows[r] & mask)) { rows[r] ^=
rows[rank]; } } ++rank; } return rank; } int main(int argc, char** argv) { // Default file name as
requested string filename = "output.dat"; if (argc >= 2) filename = argv[1]; ifstream fin(filename); if
(!fin) { cerr << "Error: cannot open file '" << filename << "'\n"; return 1; } // Original Diehard uses
40,000 matrices for rank32 test const int M_target = 40000; long long N32 = 0, N31 = 0, NLE30 = 0;
int matrices_done = 0; for (; matrices_done < M_target; ++matrices_done) { uint32_t rows[32]; //
build one 32x32 matrix from 128 bytes (32 words) for (int r = 0; r < 32; ++r) { uint8_t b[4]; for (int k =
0; k < 4; ++k) { if (!read_hex_byte(fin, b[k])) { cerr << "EOF: not enough hex bytes to complete matrix
" << (matrices_done + 1) << "\n"; goto done; } } rows[r] = bytes_to_u32(b); } int rank =
gf2_rank_32(rows); if (rank == 32) ++N32; else if (rank == 31) ++N31; else ++NLE30; } done: long long
M = N32 + N31 + NLE30; if (M == 0) { cerr << "No matrices processed.\n"; return 1; } // Expected
counts double E32 = (double)M * P32; double E31 = (double)M * P31; double ELE30 = (double)M *
PLE30; // Chi-square (df=2) double chi2 = ((N32 - E32) * (N32 - E32)) / E32 + ((N31 - E31) * (N31 -
E31)) / E31 + ((NLE30 - ELE30) * (NLE30 - ELE30)) / ELE30; // For df=2, p-value = exp(-chi2/2) double
p_value = exp(-0.5 * chi2); cout << "Binary Rank 32x32 (Diehard-style)\n"; cout << "Input file: " <<
filename << "\n"; cout << "Matrices processed M = " << M << "\n\n"; cout << "Observed counts:\n";
cout << " N(rank=32) = " << N32 << "\n"; cout << " N(rank=31) = " << N31 << "\n"; cout << "
N(rank<=30) = " << NLE30 << "\n\n"; cout << "Expected counts (Diehard):\n"; cout << " E32 = " <<
E32 << "\n"; cout << " E31 = " << E31 << "\n"; cout << " E<=30 = " << ELE30 << "\n\n"; cout << "chi^2
(df=2) = " << chi2 << "\n"; cout << "p-value = " << p_value << "\n"; if (p_value < 1e-6 || p_value > 1.0
- 1e-6) { cout << "WARNING: Extreme p-value (very close to 0 or 1) is suspicious.\n"; } // Practical hint
about endianness cout << "\nNote: WORD_LITTLE_ENDIAN = " << (WORD_LITTLE_ENDIAN ? "true" :
"false") << "\n"; cout << "If results look unreasonable, flip WORD_LITTLE_ENDIAN and rerun.\n";
return 0; }

```

You said:

anything left that i missed in it

ChatGPT said:

A few important things you likely still need (or to double-check) so your implementation matches “real Diehard” behavior and gives trustworthy results:

1) Your file format assumption

Your code reads **hex bytes in a text file** (tokens like AF 01 2C ...).

Original Diehard typically reads a **binary file** of 32-bit integers.

So you’re fine *if* your output.dat truly is hex-text representing exactly **5,120,000 bytes**.

If instead it’s a raw binary .dat, this reader will be wrong.

Quick check: open output.dat in a text editor—if it’s readable hex tokens, OK.

2) Endianness / bit order

Even with correct bytes, the **byte order inside each 32-bit word** matters:

- little-endian vs big-endian will change rank counts.
- Diehard on PCs historically uses the machine’s natural word order (often little-endian).

You already have WORD_LITTLE_ENDIAN. Good.

But also note: **bit significance (MSB/LSB) doesn’t matter for rank**, only the word’s bits as a set, so you’re safe there.

3) Exactly 40,000 matrices

Original Diehard does **exactly 40,000** for 32×32 rank.

Your code:

- tries up to 40,000,
- but if EOF occurs early, it stops and reports with smaller M.

That’s correct and robust. Just make sure your data really contains enough tokens:

- need **40,000 × 128 = 5,120,000 bytes**
 - since hex text expands, the file size in characters will be much larger than 5.12 MB.
-

4) Correct p-value formula

For **df = 2**, the chi-square survival function is:

$$p = e^{-\chi^2/2}$$

So your `exp(-0.5*chi2)` is correct **only because df=2**.

If you later add other tests with different df, you’ll need a general chi-square CDF/survival.

5) Don't forget the "suite logic"

Diehard doesn't just look at one p-value; it expects many tests and you judge failures by:

- repeated extreme p-values,
- patterns across tests,
- and across different parts of the stream.

So what's "left" is: run **multiple tests** and/or run each test on different segments (windows) and collect p-values.

6) Speed considerations for hex-text input

The rank computation is fast; the slow part will be parsing hex tokens.

If you want more speed later:

- convert hex-text → binary once (a converter program),
- then read binary `uint32_t` directly.

That will be ~10–50× faster on large files.

7) A small but useful output addition

Add printing of **expected vs observed proportions** (N/M) to eyeball sanity quickly:

- N31 should be about ~0.58M
- N32 about ~0.29M
- N≤30 about ~0.13M

Not required, but makes debugging easier.

Bottom line

For *this single test*, you didn't miss anything fundamental **provided** your input truly is **hex bytes (text)** and your endianness is correct.

If you paste the first ~20–40 hex tokens from `output.dat`, I can tell you whether the file looks like "byte stream" and which endianness setting is more likely correct.

You said:

u mean for this test i need integer file not a hex file

ChatGPT said:

No — you **don't strictly need** an "integer file." What you need is a stream of **bits**. Original Diehard *happened* to read them as **32-bit integers from a binary file** (because that's fast/easy on C). If your data is stored as **hex text**, that's still the *same bits*, just encoded as text.

What original Diehard expects

- A **binary** file (raw bytes), read as unsigned long / 32-bit words.
- For Rank 32×32 it consumes **5,120,000 raw bytes**.

What you have

- A .dat file where values are written in **hex text** (e.g., AF 01 2C ...).
- That's OK **if** it represents exactly the same 5,120,000 bytes (just printed as hex).

The difference (important)

- **Binary file:** 5,120,000 bytes on disk.
- **Hex-text file:** much larger on disk (because each byte becomes 2 hex chars + space/newline), but it *represents* 5,120,000 bytes.

So your code is fine if:

1. Your file is truly hex byte tokens (00–FF), and
2. It contains at least **5,120,000 bytes worth of tokens** (i.e., 5,120,000 hex pairs).

If you want to match Diehard “exactly” (recommended)

Convert your hex-text into a **raw binary** file once, then run the rank test on that binary file. It will be faster and closer to original Diehard behavior.

If you tell me which format your output.dat is:

- Example A (hex text): 3F A2 00 FF ...
- Example B (binary): unreadable garbage in a text editor

...I'll give you the correct C++ reader version (hex-text vs raw-binary) and, if needed, a tiny converter.

You said:

now in first row i have to write first 32 bits so i have 12, af,d3,56 now how will first row will be

ChatGPT said:

You take those **4 bytes** and just write their bits in order to make **one 32-bit row**.

If you place bytes in the same order you wrote them (big-endian view)

Bytes: 12 AF D3 56

- 0x12 = 00010010
- 0xAF = 10101111
- 0xD3 = 11010011
- 0x56 = 01010110

So the **first row (32 bits)** is:

00010010 10101111 11010011 01010110

(= 0x12AFD356)

If you interpret the same 4 bytes as a little-endian 32-bit word in memory

The 32-bit value would be 0x56D3AF12, and the row written MSB→LSB becomes:

01010110 11010011 10101111 00010010

Key point (so you don't worry)

For the **rank test**, swapping byte order or reversing bit order just **permutes columns**, and **rank does not change** under column permutations. So either representation is fine as long as you're consistent.

You said:

so how to find rank of this matrix 32x32 step to step

ChatGPT said:

To find the **rank of a 32×32 bit matrix** in Diehard, you do **Gaussian elimination over GF(2)** (binary field). That means:

- **Addition = XOR**
- **Subtraction = XOR**
- No fractions, no division—only XOR row operations.

Below is the step-by-step process you follow for **any 32×32**, including the one you build from your bytes.

Setup

You already formed your matrix *A* with 32 rows, each row is 32 bits.

In code (fast way), store each row as a **uint32_t**:

- Row *i* is a 32-bit integer rows[i].

Example: your first row bytes 12 AF D3 56 could be stored as:

- big-endian word: 0x12AFD356
(or little-endian 0x56D3AF12—rank won't change if you're consistent.)

Goal

Convert the matrix to a “row-reduced” form using XOR, and count pivots.

The number of pivots you manage to create = **rank**.

Step-by-step elimination (exact Diehard logic)

Step 1: Start

- rank = 0
- You'll try to create pivots one by one.

Step 2: Choose a pivot column

Pick a column to eliminate on. A common choice (used in fast implementations) is:

- start from **MSB column 31** down to **LSB column 0**.

So loop: col = 31, 30, ..., 0.

Define a bit mask for this column:

- mask = 1u << col

Step 3: Find a pivot row

Look for a row r at or below the current pivot position rank such that:

- rows[r] & mask is nonzero → means that row has a **1** in this column.

If you **cannot find** such a row:

- this column can't give a pivot → move to next column.

Step 4: Swap pivot row into position

If you found pivot row pivot and it is not already at rows[rank]:

- swap rows[pivot] and rows[rank].

Now rows[rank] is the pivot row.

Step 5: Eliminate the pivot column from other rows

For every other row i (0..31, i != rank):

- if rows[i] has a 1 in the pivot column (rows[i] & mask):
 - do: rows[i] ^= rows[rank]

This XOR removes the 1 in that column (because 1 XOR 1 = 0).

Step 6: Increase rank

- rank++
- move to the next column and repeat.

Step 7: Final answer

After finishing columns, rank is the matrix rank over GF(2).

- If rank = 32 → full rank
 - If rank = 31 → one dependency
 - If rank ≤ 30 → more dependencies
-

Tiny “toy” example (so the XOR idea is clear)

Suppose 4-bit rows (just to show mechanics):

Row A: 1011

Row B: 1001

Pivot on leftmost bit (bit3 = 1). Row A is pivot.

Eliminate from Row B:

- RowB has that bit = 1, so:
 - RowB = RowB XOR RowA
 - 1001 XOR 1011 = 0010

Now Row B no longer has the pivot bit set.

Same concept, but with 32-bit rows.

Practical note (important)

Doing elimination **row-wise** with `uint32_t` is why this test is fast:

- each XOR eliminates a whole 32-bit row in one CPU instruction (very fast).
-

If you want, paste **your first 4–6 rows** (each as 8 hex bytes = 32 bits) and I’ll walk through the first few pivot/elimination steps explicitly (showing swaps and XOR results) so you can see rank building in real numbers.

You said:

can u explain it b example step to step

ChatGPT said:

Yes—let’s do a **fully worked example** step-by-step. I’ll use a **small matrix (4×4)** so you can see every move. The **32×32 test is identical**, just bigger (same XOR row-operations).

Rule (GF(2) math)

- Add/subtract rows = **XOR**
 - Goal: make “pivot 1s” and clear the column using XOR
 - **Rank = number of pivots**
-

Example matrix (4×4 bits)

Suppose the RNG gave these 4 rows (each row is 4 bits):

- R1 = 1 0 1 1

- $R2 = 1\ 1\ 0\ 1$
- $R3 = 0\ 1\ 1\ 1$
- $R4 = 1\ 1\ 1\ 0$

Write them compactly:

- $R1 = \mathbf{1011}$
- $R2 = \mathbf{1101}$
- $R3 = \mathbf{0111}$
- $R4 = \mathbf{1110}$

We will pivot left $\rightarrow$ right (column 1 to 4). Same as 32×32 .

Step 1: Pivot in Column 1 (leftmost bit)

Column 1 has 1s in R1, R2, R4. Choose R1 as pivot.

Pivot row = $R1 = 1011$.

Now eliminate column 1 from all other rows that have a 1 in column 1.

Eliminate R2 (since it starts with 1)

$R2 = R2 \text{ XOR } R1$

$1101 \text{ XOR } 1011 = 0110$

So R2 becomes **0110**.

Eliminate R4 (since it starts with 1)

$R4 = R4 \text{ XOR } R1$

$1110 \text{ XOR } 1011 = 0101$

So R4 becomes **0101**.

R3 starts with 0 $\rightarrow$ do nothing.

Now matrix is:

- $R1 = 1011$ (pivot col1)
- $R2 = 0110$
- $R3 = 0111$
- $R4 = 0101$

We created 1 pivot $\Rightarrow$ rank = 1 so far.

Step 2: Pivot in Column 2

Now look at remaining rows below pivot (R2, R3, R4) in column 2:

- R2 starts with 0 1 ... → column2 is 1 ✓
So choose R2 as pivot.

Pivot row = R2 = 0110.

Now eliminate column 2 from other rows that have 1 in column 2 (R3 and R4).

Eliminate R3

R3 = R3 XOR R2
0111 XOR 0110 = 0001

So R3 becomes **0001**.

Eliminate R4

R4 = R4 XOR R2
0101 XOR 0110 = 0011

So R4 becomes **0011**.

R1 has column2=0 → no change.

Now:

- R1 = 1011
- R2 = 0110 (pivot col2)
- R3 = 0001
- R4 = 0011

Second pivot ⇒ rank = 2.

Step 3: Pivot in Column 3

Look at rows below (R3, R4) in column 3:

- R3 = 0001 → column3 is 0
- R4 = 0011 → column3 is 1 ✓

So pivot must be R4. Swap R3 and R4 (because pivot should be next row).

After swap:

- R3 = 0011 (pivot col3)
- R4 = 0001

Now eliminate column 3 from other rows that have 1 in column 3.

Check:

- R1 = 1011 → column3 is 1 ✓
- R2 = 0110 → column3 is 1 ✓

- $R4 = 0001 \rightarrow \text{column3 is } 0$

Eliminate R1

$$R1 = R1 \text{ XOR } R3$$

$$1011 \text{ XOR } 0011 = 1000$$

R1 becomes **1000**.

Eliminate R2

$$R2 = R2 \text{ XOR } R3$$

$$0110 \text{ XOR } 0011 = 0101$$

R2 becomes **0101**.

Now:

- $R1 = 1000$
- $R2 = 0101$
- $R3 = 0011$ (pivot col3)
- $R4 = 0001$

Third pivot $\Rightarrow$ rank = 3.

Step 4: Pivot in Column 4

Look at row below (R4) in column 4:

- $R4 = 0001 \rightarrow \text{column4 is } 1$ ☒

Pivot row = R4.

Eliminate column 4 from rows above that have 1 in column4:

- $R2 = 0101 \rightarrow \text{has col4}=1$ ☒
- $R3 = 0011 \rightarrow \text{has col4}=1$ ☒
- $R1 = 1000 \rightarrow \text{col4}=0$

Eliminate R2

$$R2 = R2 \text{ XOR } R4$$

$$0101 \text{ XOR } 0001 = 0100$$

Eliminate R3

$$R3 = R3 \text{ XOR } R4$$

$$0011 \text{ XOR } 0001 = 0010$$

Now:

- $R1 = 1000$

- R2 = 0100
- R3 = 0010
- R4 = 0001

This is basically an identity matrix (all pivots).

Fourth pivot $\Rightarrow$ rank = 4 (full rank).

Final Result

☒ Rank = 4 (full rank)

How this maps to 32×32

Exactly the same, just:

- rows are **32-bit**
- columns are **32**
- XOR operations are on **uint32_t**
- rank can be 0...32
- Diehard only counts: rank 32, rank 31, rank ≤ 30

If you paste **just 3–5 of your real 32-bit rows** (for example: 12AFD356, A0C1..., etc.), I can show you the **first pivot + first few XOR eliminations** exactly like above—using your data.

You said:

can u write a snippet code for it in c++ for din rank in 32x32

ChatGPT said:

```
// GF(2) rank of a 32x32 binary matrix. // Represent the matrix as 32 rows, each row packed into a
uint32_t. // Bit operations are over GF(2): elimination uses XOR. // // rank = number of pivots found
(0..32). // // Usage: // uint32_t rows[32] = {...}; // fill with your 32 words // int r = rank_gf2_32(rows);
// NOTE: modifies rows in-place // #include <cstdint> using namespace std; int rank_gf2_32(uint32_t
rows[32]) { int rank = 0; // Pivot from MSB (bit 31) down to LSB (bit 0) for (int col = 31; col >= 0 &&
rank < 32; --col) { uint32_t mask = (1u << col); // 1) Find pivot row at/below current rank with a 1 in
this column int pivot = -1; for (int r = rank; r < 32; ++r) { if (rows[r] & mask) { pivot = r; break; } } if
(pivot == -1) continue; // no pivot in this column // 2) Swap pivot row into position "rank" if (pivot !=
rank) { uint32_t tmp = rows[pivot]; rows[pivot] = rows[rank]; rows[rank] = tmp; } // 3) Eliminate this
column from all other rows for (int r = 0; r < 32; ++r) { if (r != rank && (rows[r] & mask)) { rows[r] ^=
rows[rank]; } } // 4) One pivot found ++rank; } return rank; }
```

Notes (so you don't get surprised):

- This modifies rows[] (row-reduction). If you need to keep original, copy it first.
- Store each matrix row as one uint32_t word (32 bits). Each word corresponds to 32 entries in that row.
- This is exactly the GF(2) Gaussian elimination used for the Diehard rank test.

You said:

now explain Binary Rank 31×31 step to step like i dont know anything

ChatGPT said:

Binary Rank **31×31** is the same idea as 32×32, just with a **31×31 bit matrix** and a slightly different way of consuming bits from the stream.

What the test is trying to detect

Bad RNGs often have **linear structure**: some bits are XOR-combinations of others.

A random 31×31 matrix should very often be full rank (31), sometimes 30, rarely ≤29. If your RNG is “too linear,” these counts shift.

Step-by-step (Diehard-style)

Step 1: Read bits from the stream and build a 31×31 matrix

A 31×31 matrix needs:

$$31 \times 31 = 961 \text{ bits}$$

Diehard reads your stream as 32-bit integers, but for this test it **uses only 31 bits per row**.

How one matrix is formed

For each of the 31 rows:

1. Read one 32-bit word x
2. Take **31 bits** from it (usually the **lowest 31 bits** by masking off the top bit):

$$\text{row} = x \& 0x7FFFFFFF$$

(0x7FFFFFFF = binary 0111...111 (31 ones))

This gives you 31 rows, each row has 31 bits → a 31×31 matrix.

Data used per matrix:

- 31 words × 32 bits = 992 bits read
- but only 961 bits are actually used (31 bits × 31 rows)

So per matrix Diehard consumes:

- **31 × 4 = 124 bytes**

Step 2: Compute the rank over GF(2)

Same elimination method:

- Row operations are XOR.
- You try to find pivot bits column-by-column.
- Rank is how many pivots you find.

Efficient representation

Store each row in a 32-bit integer but only 31 bits are meaningful:

- use uint32_t row with top bit zero.

Elimination process (conceptual)

- Start rank = 0
- For each column from bit 30 down to bit 0:
 - find a row at/below rank with a 1 in that column
 - swap into pivot position
 - XOR pivot row into other rows that also have 1 in that column
 - rank++

End: rank is between 0 and 31.

Step 3: Repeat many matrices and count ranks

Diehard groups ranks into **three bins** (same style as 32×32):

- N_{31} : rank = 31
- N_{30} : rank = 30
- $N_{\leq 29}$: rank ≤ 29

Total matrices:

$$M = N_{31} + N_{30} + N_{\leq 29}$$

Step 4: Compare to theoretical probabilities

For a truly random 31×31 binary matrix, the rank distribution is known (Diehard uses fixed probabilities, like for 32×32). Then compute expected counts:

$$E_{31} = MP_{31}, E_{30} = MP_{30}, E_{\leq 29} = MP_{\leq 29}$$

(If you want, I'll give you the exact Diehard constants used for 31×31—same idea as the 32×32 constants.)

Step 5: Chi-square and p-value

Compute:

$$\chi^2 = \sum \frac{(N_i - E_i)^2}{E_i}$$

with

- 3 bins → **df = 2**
Then p-value from chi-square (df=2).
-

What changes from 32×32?

1. **Matrix size:** 31×31 instead of 32×32
 2. **Bits per row:** only 31 bits used (& 0x7FFFFFFF)
 3. **Bytes per matrix:** 124 bytes instead of 128 bytes
 4. **Bins:** rank 31 / 30 / ≤29 instead of 32 / 31 / ≤30
Everything else (GF(2) elimination, chi-square) is identical.
-

If you want, I'll also write the **C++ rank function for 31×31** (it's basically the same as your 32×32 but loops only 31 rows/columns and masks bits).

You said:

what is expected value in it

ChatGPT said:

Thought for 15s

In the **original Diehard 31×31 Binary Rank test**, you count **4 rank categories**:

- rank = **31**
- rank = **30**
- rank = **29**
- rank ≤ **28** (all ranks 0–28 pooled together, because very small ranks are rare)

Diehard uses **40,000 matrices**. For truly random bits, the **expected counts** (the “expected value” per bin) are:

Expected counts for M = 40,000

Rank bin	Expected count E
31	11551.5
30	23103.0
29	5134.0
≤ 28	211.4

These correspond to probabilities (just divide by 40,000):

- $P(31) \approx 11551.5/40000 \approx 0.2887875$
- $P(30) \approx 23103.0/40000 \approx 0.5775750$
- $P(29) \approx 5134.0/40000 \approx 0.1283500$
- $P(\leq 28) \approx 211.4/40000 \approx 0.0052850$

What you do with these “expected values”

After you compute your observed counts $N_{31}, N_{30}, N_{29}, N_{\leq 28}$, you do a chi-square:

$$\chi^2 = \sum \frac{(N_i - E_i)^2}{E_i}$$

Because there are **4 bins**, degrees of freedom is:

$$df = 4 - 1 = 3$$

Then you convert χ^2 (df=3) to a p-value.

If you want, I can give you the exact C++ constants for these 4 probabilities and the rank-31×31 implementation (masking to 31 bits + GF(2) elimination) matching this binning.

You said:

in 32×32 binary test how munk rankFor 32×32, the theoretical probabilities are approximately0
When the test finishes looking at all 40,000 matrices, it puts the results into exactly four buckets:
Rank 32: The "Full Rank" bucket. but u take only 32 31 ,<=30 Rank 31: The "One Less" bucket. Rank
30: The "Two Less" bucket. Rank < 30: The "Everything Else" bucket.

ChatGPT said:

You’re right to notice the mismatch — **the “original Diehard” 32×32 rank test does *not* use 4 buckets. It uses 3 buckets:**

- **Rank = 32**

- **Rank = 31**
- **Rank ≤ 30** (*this includes rank 30, 29, ..., 0 all together*)

So in **original Diehard**, there is **no separate “Rank 30” bucket** for the 32×32 test.

Why Diehard groups it as ≤ 30 (instead of splitting 30 and <30)

Because for a truly random 32×32 binary matrix, ranks below 31 are relatively rare, and Diehard wants each bucket to have a **reasonably large expected count** for a reliable chi-square test. So it combines everything ≤ 30 into one bin.

What happens to “Rank 30” probability?

It's **included inside** the probability for **Rank ≤ 30** .

So when we say (for 32×32):

- $P(32) \approx 0.288788095$
- $P(31) \approx 0.577576190$
- $P(\leq 30) \approx 0.133635715$

that last number already includes rank 30 **and** all smaller ranks.

Where does the “4 buckets” description come from?

Some **variants / explanations** (and some other rank tests like **31×31**) use **4 bins** (e.g., exact ranks 31, 30, 29, ≤ 28). That's **not** the classic 32×32 Diehard binning.

For your implementation

If you want to match **original Diehard 32×32** , keep exactly:

- `rank==32` $\rightarrow$ N32
- `rank==31` $\rightarrow$ N31
- `else` $\rightarrow$ NLE30

If you want, I can also show you how to implement a **4-bucket** version (32,31,30, ≤ 29), but that would no longer be “original Diehard” and you'd need the correct theoretical probabilities for those bins.

You said:

in binary 31×31 how much buckete use in originakdiehars test

ChatGPT said:

Thought for 8s

In the **original Diehard Binary Rank 31×31** test, it uses **4 buckets (bins)** for the chi-square:

1. **Rank = 31**
2. **Rank = 30**
3. **Rank = 29**

4. Rank ≤ 28 (all ranks 0–28 pooled together)

That's exactly how the classic Diehard-style output/report is formatted for 31x31.

You said:

now give .cpp code for 31x31 same type output.dat file

ChatGPT said:

```
// diehard_rank31.cpp // // Binary Rank 31x31 test (original Diehard-style binning: 31, 30, 29, <=28).  
// INPUT : output.dat (TEXT file containing HEX BYTES, e.g. "0A FF 1c ...") // OUTPUT : counts + chi-  
square + p-value // // Matrix construction (Diehard style): // - Build 31 rows. Each row is taken from  
ONE 32-bit word but ONLY 31 bits are used. // - We mask the top bit off: row = word & 0x7FFFFFFF. //  
// Recommended M in classic Diehard is 40,000 matrices. // // Compile: // g++ -O3 -std=c++17  
diehard_rank31.cpp -o diehard_rank31 // // Run (reads output.dat by default): // ./diehard_rank31  
// Or: // ./diehard_rank31 output.dat // // NOTE: If your 32-bit word byte-order is different, flip  
WORD_LITTLE_ENDIAN. #include <cstdlib> #include <stdlib.h> #include <cmath> #include <iostream>  
#include <fstream> #include <string> using namespace std; static constexpr bool  
WORD_LITTLE_ENDIAN = true; // set false if your word order is big-endian // Diehard expected  
probabilities for 31x31 bins: rank 31, 30, 29, <=28 // These correspond to expected counts for  
M=40000 shown in classic Diehard writeups. // E31=11551.5, E30=23103.0, E29=5134.0,  
E<=28=211.4 static constexpr double P31 = 11551.5 / 40000.0; static constexpr double P30 = 23103.0  
/ 40000.0; static constexpr double P29 = 5134.0 / 40000.0; static constexpr double PLE28 = 211.4 /  
40000.0; // Read next hex byte token from stream (robust to separators, optional 0x prefix). static  
bool read_hex_byte(istream& in, uint8_t& outByte) { string tok; while (in >> tok) { while  
(!tok.empty() && (tok.back() == ',' || tok.back() == ';' || tok.back() == ':')) tok.pop_back(); if  
(tok.empty()) continue; if (tok.size() >= 2 && tok[0] == '0' && (tok[1] == 'x' || tok[1] == 'X')) { tok =  
tok.substr(2); } if (tok.empty()) continue; char* endp = nullptr; long v = strtoul(tok.c_str(), &endp, 16);  
if (endp == tok.c_str() || *endp != '\0' || v < 0 || v > 255) { continue; } outByte =  
static_cast<uint8_t>(v); return true; } return false; } static uint32_t bytes_to_u32(const uint8_t b[4]) {  
if (WORD_LITTLE_ENDIAN) { return (uint32_t)b[0] | ((uint32_t)b[1] << 8) | ((uint32_t)b[2] << 16) |  
((uint32_t)b[3] << 24); } else { return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) | ((uint32_t)b[2]  
<< 8) | (uint32_t)b[3]; } } // Rank of 31x31 binary matrix over GF(2). // rows[0..30] each has only 31  
active bits (bit 30..0). Bit 31 is 0. // NOTE: modifies rows in-place. static int gf2_rank_31(uint32_t  
rows[31]) { int rank = 0; // pivot from bit 30 down to bit 0 (31 columns) for (int col = 30; col >= 0 &&  
rank < 31; --col) { uint32_t mask = (1u << col); int pivot = -1; for (int r = rank; r < 31; ++r) { if (rows[r] &  
mask) { pivot = r; break; } } if (pivot < 0) continue; if (pivot != rank) { uint32_t tmp = rows[pivot];  
rows[pivot] = rows[rank]; rows[rank] = tmp; } for (int r = 0; r < 31; ++r) { if (r != rank && (rows[r] &  
mask)) { rows[r] ^= rows[rank]; } } ++rank; } return rank; } int main(int argc, char** argv) { string  
filename = "output.dat"; if (argc >= 2) filename = argv[1]; ifstream fin(filename); if (!fin) { cerr <<  
"Error: cannot open file '" << filename << "'\n"; return 1; } const int M_target = 40000; long long N31  
= 0, N30 = 0, N29 = 0, NLE28 = 0; int matrices_done = 0; for (; matrices_done < M_target;  
++matrices_done) { uint32_t rows[31]; // For each row: read one 32-bit word (4 bytes) then keep  
only 31 bits for (int r = 0; r < 31; ++r) { uint8_t b[4]; for (int k = 0; k < 4; ++k) { if (!read_hex_byte(fin,  
b[k])) { cerr << "EOF: not enough hex bytes to complete matrix " << (matrices_done + 1) << "\n"; goto  
done; } } uint32_t w = bytes_to_u32(b); rows[r] = (w & 0x7FFFFFFFu); // keep only 31 bits } int rank =  
gf2_rank_31(rows); if (rank == 31) ++N31; else if (rank == 30) ++N30; else if (rank == 29) ++N29; else  
++NLE28; } done: long long M = N31 + N30 + N29 + NLE28; if (M == 0) { cerr << "No matrices  
processed.\n"; return 1; } // Expected counts double E31 = (double)M * P31; double E30 = (double)M
```



```

* P30; double E29 = (double)M * P29; double ELE28 = (double)M * PLE28; // Chi-square (4 bins =>
df=3) double chi2 = ((N31 - E31) * (N31 - E31)) / E31 + ((N30 - E30) * (N30 - E30)) / E30 + ((N29 - E29)
* (N29 - E29)) / E29 + ((NLE28 - ELE28) * (NLE28 - ELE28)) / ELE28; // p-value for chi-square(df=3): use
survival function via incomplete gamma. // We'll use a simple approximation by calling std::erfc for
df=1/2 only is not enough. // Instead, we compute p-value using regularized gamma Q(k/2, chi2/2)
with k=3. // // For df=3, Q(1.5, x) has a closed form: // Q(3/2, x) = erfc(sqrt(x)) + (2/sqrt(pi)) * sqrt(x)
* exp(-x) // double x = 0.5 * chi2; // x = chi2/2 double sx = sqrt(x); double p_value = erfc(sx) + (2.0 /
sqrt(M_PI)) * sx * exp(-x); cout << "Binary Rank 31x31 (Diehard-style)\n"; cout << "Input file: " <<
filename << "\n"; cout << "Matrices processed M = " << M << "\n\n"; cout << "Observed counts:\n";
cout << " N(rank=31) = " << N31 << "\n"; cout << " N(rank=30) = " << N30 << "\n"; cout << "
N(rank=29) = " << N29 << "\n"; cout << " N(rank<=28) = " << NLE28 << "\n\n"; cout << "Expected
counts (Diehard):\n"; cout << " E31 = " << E31 << "\n"; cout << " E30 = " << E30 << "\n"; cout << " E29
= " << E29 << "\n"; cout << " E<=28 = " << ELE28 << "\n\n"; cout << "chi^2 (df=3) = " << chi2 << "\n";
cout << "p-value = " << p_value << "\n"; if (p_value < 1e-6 || p_value > 1.0 - 1e-6) { cout <<
"WARNING: Extreme p-value (very close to 0 or 1) is suspicious.\n"; } cout << "\nNote:
WORD_LITTLE_ENDIAN = " << (WORD_LITTLE_ENDIAN ? "true" : "false") << "\n"; cout << "If results
look unreasonable, flip WORD_LITTLE_ENDIAN and rerun.\n"; return 0; }

```

You said:

also gibe binary 32x32 count of expected value of rank 32 31 like that

ChatGPT said:

Thought for 11s

For the **original Diehard Binary Rank 32×32** test (with **M = 40,000** matrices) the bins are:

- **Rank = 32**
- **Rank = 31**
- **Rank ≤ 30** (all 30,29,...,0 together)

Using Diehard's theoretical probabilities:

- $P(32) = 0.288788095$
- $P(31) = 0.577576190$
- $P(\leq 30) = 0.133635715$

Expected counts (M = 40,000)

$$E_i = M P_i$$

- **Expected Rank 32:**

$$E_{32} = 40000 \times 0.288788095 = 11551.5238 \approx \mathbf{11551.5}$$

- **Expected Rank 31:**

$$E_{31} = 40000 \times 0.577576190 = 23103.0476 \approx \mathbf{23103.0}$$

- **Expected Rank ≤ 30:**

$$E_{\leq 30} = 40000 \times 0.133635715 = 5345.4286 \approx \mathbf{5345.4}$$

Check: $11551.5238 + 23103.0476 + 5345.4286 = 40000$.

You said:

what can be observed value for 32x32 and 31x31

ChatGPT said:

“Observed values” here mean the **counts you actually measure from your data** after forming the matrices and computing ranks. There is no single fixed observed value—it **depends on your RNG/data**. But for a *good* RNG, the observed counts should be **close to the expected counts**, within normal random fluctuation.

Below are the expected counts and the **typical range** you’d see (roughly) if your data is truly random and you run the original Diehard sample size **M = 40,000**.

1) Binary Rank 32×32 (Diehard bins: 32, 31, ≤30)

Expected (M=40,000)

- $E_{32} \approx 11551.5$
- $E_{31} \approx 23103.0$
- $E_{\leq 30} \approx 5345.4$

Typical observed ranges (about “ $\pm 3\sigma$ ” rule-of-thumb)

Use Binomial SD: $\sigma \approx \sqrt{Mp(1-p)}$

- **Rank 32:** $p = 0.288788$
 $\sigma \approx \sqrt{40000 \cdot 0.2888 \cdot 0.7112} \approx 90.7$
Typical: $11552 \pm 3(91) \rightarrow \sim \mathbf{11280 \text{ to } 11820}$
- **Rank 31:** $p = 0.577576$
 $\sigma \approx \sqrt{40000 \cdot 0.5776 \cdot 0.4224} \approx 98.7$
Typical: $23103 \pm 3(99) \rightarrow \sim \mathbf{22810 \text{ to } 23395}$
- **Rank ≤30:** $p = 0.133636$
 $\sigma \approx \sqrt{40000 \cdot 0.1336 \cdot 0.8664} \approx 68.0$
Typical: $5345 \pm 3(68) \rightarrow \sim \mathbf{5140 \text{ to } 5550}$

These aren’t strict pass/fail limits—just what you usually see.

2) Binary Rank 31×31 (Diehard bins: 31, 30, 29, ≤28)

Diehard’s classic expected counts for **M = 40,000** are:

- $E_{31} = 11551.5$
- $E_{30} = 23103.0$
- $E_{29} = 5134.0$

- $E_{\leq 28} = 211.4$

Convert to probabilities (divide by 40,000):

- $p_{31} \approx 0.2887875$
- $p_{30} \approx 0.5775750$
- $p_{29} \approx 0.12835$
- $p_{\leq 28} \approx 0.005285$

Typical observed ranges ($\pm 3\sigma$)

- **Rank 31:** same as above $\rightarrow \sigma \approx 90.7$
~ **11280 to 11820**
- **Rank 30:** same as above $\rightarrow \sigma \approx 98.7$
~ **22810 to 23395**
- **Rank 29:** $p \approx 0.12835$

$$\sigma \approx \sqrt{40000 \cdot 0.12835 \cdot 0.87165} \approx 66.9$$

Typical: $5134 \pm 3(67) \rightarrow \sim \mathbf{4933 \text{ to } 5335}$

- **Rank ≤ 28 :** $p \approx 0.005285$

$$\sigma \approx \sqrt{40000 \cdot 0.005285 \cdot 0.994715} \approx 14.5$$

Typical: $211 \pm 3(15) \rightarrow \sim \mathbf{168 \text{ to } 255}$

How to use this

1. Run your code $\rightarrow$ get observed counts.
2. If counts are wildly outside these rough ranges, you'll usually get an extreme chi-square p-value.
3. The real decision is the **p-value** (and patterns across multiple tests/windows), not just one bin.

If you paste your **N32, N31, N $\leq$ 30** (and for 31 $\times$ 31: **N31, N30, N29, N $\leq$ 28**) I can compute chi-square and interpret whether the result looks normal.

You said:

now one more doubt in 31 $\times$ 31 in each row which 31 bit should take lsb or msb

ChatGPT said:

In the **original Diehard 31 $\times$ 31 rank test**, you take the **31 least-significant bits (LSB 31 bits)** of each 32-bit word.

Exactly what that means

For each 32-bit integer x read from the stream, form the row as:

`row = x & 0x7FFFFFFF; // keep bits 0..30 (LSB 31 bits), clear bit 31 (MSB)`

So you **drop the top (most-significant) bit** and keep the lower 31 bits.

Why LSB (and not MSB)?

Diehard's implementation is based on a simple mask operation ($\& 0x7FFFFFFF$) after reading a 32-bit word, which naturally keeps the **lowest 31 bits**.

If I accidentally take MSB 31 bits instead, will rank change?

Usually rank won't change in a systematic way for a good RNG, but to match **original Diehard exactly**, use **LSB 31 bits** (mask with $0x7FFFFFFF$).

You said:

in 32x32 in column how to take bits left to right or right to left

ChatGPT said:

For the **32x32 rank test**, it **does not matter** whether you treat columns as **left→right (MSB→LSB)** or **right→left (LSB→MSB)**.

Why it doesn't matter (key fact)

Changing "left-to-right vs right-to-left" is just **relabeling / permuting columns** (a column permutation).

Rank is invariant under column permutations, so the computed rank is identical.

What Diehard-style implementations typically do

Most fast C/C++ implementations pivot on bits in one of these two equivalent ways:

Option A (MSB → LSB):

- column order: bit 31, 30, ..., 0
- mask: $1u \ll \text{col}$ with col decreasing

Option B (LSB → MSB):

- column order: bit 0, 1, ..., 31
- mask: $1u \ll \text{col}$ with col increasing

Both give the **same rank** for every matrix.

What you should do

Pick one convention and be consistent across all matrices.

Your earlier code uses **MSB→LSB**, which is perfectly fine.

If you later compare with someone else's output and want the exact same intermediate reduced matrix (not just rank), then you'd match their bit order—but for Diehard rank counts and p-values, it's irrelevant.

You said:

now i have read both 32x32 and 31x31 now explain both step to step and all information also add how much expected and in end write both codes and also all information

ChatGPT said:

Below is a complete, **from-scratch** explanation of **both** Diehard Binary Rank tests (**32×32** and **31×31**) + **expected counts** (original Diehard, $M = 40000$) + **both C++ codes** for an output.dat file that contains **hex bytes as text**.

A. What “Binary Rank” tests are checking (intuition)

1. Take bits from your RNG and arrange them into a square matrix of 0/1.
 2. Compute the **rank over GF(2)** (binary arithmetic):
 - add/subtract = **XOR**
 3. A good RNG makes matrices that behave like random binary matrices, so the ranks follow a known distribution.
 4. If your RNG has **linear structure** (common weakness), you get **too many low-rank matrices**, shifting counts and giving an extreme p-value.
-

B. GF(2) Rank: how rank is computed (core idea)

You do **Gaussian elimination**, but using XOR:

Elimination steps (works for both tests)

- Store each row as an integer (bit-packed).
- For each column (bit position):
 1. Find a row with a 1 in that column (pivot).
 2. Swap it into the next pivot position.
 3. XOR it into other rows that have 1 in that column to clear that column.
- Number of pivots found = **rank**.

Column order MSB→LSB or LSB→MSB does not matter for rank.

1) Diehard Binary Rank 32×32 (original)

1.1 Data usage (original Diehard)

- Matrices: **M = 40,000**
- Each matrix: **32 rows × 32 bits = 1024 bits = 128 bytes**
- Total bytes needed (raw):
 $40000 \times 128 = 5,120,000$ bytes

1.2 How to build one 32×32 matrix (step-by-step)

For each matrix:

1. Read **128 bytes** from the stream.
2. Convert them into **32 words (32-bit)**.
3. Each 32-bit word becomes one row (32 bits).

So each matrix has rows[0..31] of type uint32_t.

1.3 Rank categories (buckets) in original Diehard (32×32)

Original Diehard uses **3 buckets**:

- Rank = **32**
- Rank = **31**
- Rank $\leq$ **30** (30,29,...,0 all together)

1.4 Expected counts for M = 40,000 (32×32)

Using classic Diehard probabilities:

- $P(32) = 0.288788095$
- $P(31) = 0.577576190$
- $P(\leq 30) = 0.133635715$

Expected values:

- $E_{32} = 40000 \times 0.288788095 = 11551.5238 \approx \mathbf{11551.5}$
- $E_{31} = 40000 \times 0.577576190 = 23103.0476 \approx \mathbf{23103.0}$
- $E_{\leq 30} = 40000 \times 0.133635715 = 5345.4286 \approx \mathbf{5345.4}$

1.5 Test statistic and p-value (32×32)

Compute chi-square with 3 buckets:

$$\chi^2 = \sum \frac{(N_i - E_i)^2}{E_i}, df = 3 - 1 = 2$$

For $df = 2$, p-value has a simple form:

$$p = \exp\left(-\frac{\chi^2}{2}\right)$$

2) Diehard Binary Rank 31×31 (original)

2.1 Data usage (how one matrix is formed)

A 31×31 matrix needs 961 bits, but Diehard forms it like this:

For each matrix:

1. Read **31** 32-bit words (so it consumes $31 \times 4 = \mathbf{124 \text{ bytes}}$).

2. For each word x , use **only 31 bits**:

- **Original Diehard uses the 31 LSBs:**

$$\text{row} = x \& 0x7FFFFFFF$$

That drops the MSB and keeps bits 0..30.

So each matrix has rows[0..30] as uint32_t with bit 31 forced to 0.

2.2 Buckets in original Diehard (31×31)

Original Diehard uses **4 buckets**:

- Rank = **31**
- Rank = **30**
- Rank = **29**
- Rank $\leq$ **28**

2.3 Expected counts for M = 40,000 (31×31)

Classic Diehard expected counts (M=40000):

- $E_{31} = 11551.5$
- $E_{30} = 23103.0$
- $E_{29} = 5134.0$
- $E_{\leq 28} = 211.4$

(These sum to 40000.)

2.4 Test statistic and p-value (31×31)

Chi-square with 4 buckets:

$$\chi^2 = \sum \frac{(N_i - E_i)^2}{E_i}, df = 4 - 1 = 3$$

For $df = 3$, you can compute p-value using the closed form:

Let $x = \chi^2/2$:

$$p = Q\left(\frac{3}{2}, x\right) = \text{erfc}(\sqrt{x}) + \frac{2}{\sqrt{\pi}} \sqrt{x} e^{-x}$$

C. Important practical notes

1. **Bit order left→right / right→left (32×32)**: rank doesn't change.
2. **31×31 row bits**: to match original Diehard, take **31 LSBs** (& 0x7FFFFFFF).

3. **Your file is hex-text:** parsing hex is slower than binary. If you later convert to binary, it'll run much faster.

D. C++ Code: Binary Rank 32×32 (hex-text output.dat)

```
// diehard_rank32.cpp #include <cstdlib> #include <cstdliblib> #include <cmath> #include <iostream>
#include <fstream> #include <string> using namespace std; static constexpr bool
WORD_LITTLE_ENDIAN = true; // Diehard probabilities for 32x32: bins 32, 31, <=30 static constexpr
double P32 = 0.288788095; static constexpr double P31 = 0.577576190; static constexpr double
PLE30 = 0.133635715; static bool read_hex_byte(istream& in, uint8_t& outByte) { string tok; while
(in >> tok) { while (!tok.empty() && (tok.back()!=';' || tok.back()!=';' || tok.back()!=':'))
tok.pop_back(); if (tok.empty()) continue; if (tok.size() >= 2 && tok[0]=='0' && (tok[1]=='x' ||
tok[1]=='X')) tok = tok.substr(2); if (tok.empty()) continue; char* endp = nullptr; long v =
strtol(tok.c_str(), &endp, 16); if (endp == tok.c_str() || *endp != '\0' || v < 0 || v > 255) continue;
outByte = (uint8_t)v; return true; } return false; } static uint32_t bytes_to_u32(const uint8_t b[4]) { if
(WORD_LITTLE_ENDIAN) { return (uint32_t)b[0] | ((uint32_t)b[1]<<8) | ((uint32_t)b[2]<<16) |
((uint32_t)b[3]<<24); } else { return ((uint32_t)b[0]<<24) | ((uint32_t)b[1]<<16) | ((uint32_t)b[2]<<8)
| (uint32_t)b[3]; } } static int rank_gf2_32(uint32_t rows[32]) { int rank = 0; for (int col = 31; col >= 0
&& rank < 32; --col) { uint32_t mask = (1u << col); int pivot = -1; for (int r = rank; r < 32; ++r) { if
(rows[r] & mask) { pivot = r; break; } } if (pivot < 0) continue; if (pivot != rank) swap(rows[pivot],
rows[rank]); for (int r = 0; r < 32; ++r) { if (r != rank && (rows[r] & mask)) rows[r] ^= rows[rank]; }
++rank; } return rank; } int main(int argc, char** argv) { string filename = "output.dat"; if (argc >= 2)
filename = argv[1]; ifstream fin(filename); if (!fin) { cerr << "Cannot open " << filename << "\n";
return 1; } const int M_target = 40000; long long N32=0, N31=0, NLE30=0; for (int m=0; m<M_target;
++m) { uint32_t rows[32]; for (int r=0; r<32; ++r) { uint8_t b[4]; for (int k=0; k<4; ++k) { if
(!read_hex_byte(fin, b[k])) { cerr << "EOF before completing matrix " << (m+1) << "\n"; goto done; } }
rows[r] = bytes_to_u32(b); } int rk = rank_gf2_32(rows); if (rk==32) ++N32; else if (rk==31) ++N31;
else ++NLE30; } done: long long M = N32 + N31 + NLE30; if (M==0) { cerr << "No matrices
processed.\n"; return 1; } double E32 = M*P32, E31 = M*P31, ELE30 = M*PLE30; double chi2 =
((N32-E32)*(N32-E32))/E32 + ((N31-E31)*(N31-E31))/E31 + ((NLE30-ELE30)*(NLE30-ELE30))/ELE30;
double p = exp(-0.5*chi2); // df=2 cout << "Binary Rank 32x32 (Diehard)\n"; cout << "M=" << M <<
"\n"; cout << "N32=" << N32 << " N31=" << N31 << " N<=30=" << NLE30 << "\n"; cout << "E32=" <<
E32 << " E31=" << E31 << " E<=30=" << ELE30 << "\n"; cout << "chi2(df=2)=" << chi2 << " p=" << p <<
"\n"; return 0; }
```

E. C++ Code: Binary Rank 31×31 (hex-text output.dat)

```
// diehard_rank31.cpp #include <cstdlib> #include <cstdliblib> #include <cmath> #include <iostream>
#include <fstream> #include <string> using namespace std; static constexpr bool
WORD_LITTLE_ENDIAN = true; // Expected counts (classic Diehard) for M=40000: static constexpr
double P31 = 11551.5 / 40000.0; static constexpr double P30 = 23103.0 / 40000.0; static constexpr
double P29 = 5134.0 / 40000.0; static constexpr double PLE28 = 211.4 / 40000.0; static bool
read_hex_byte(istream& in, uint8_t& outByte) { string tok; while (in >> tok) { while (!tok.empty() &&
(tok.back()!=';' || tok.back()!=';' || tok.back()!=':')) tok.pop_back(); if (tok.empty()) continue; if
(tok.size() >= 2 && tok[0]=='0' && (tok[1]=='x' || tok[1]=='X')) tok = tok.substr(2); if (tok.empty())
continue; char* endp = nullptr; long v = strtol(tok.c_str(), &endp, 16); if (endp == tok.c_str() || *endp
```



```

!= '\0' || v < 0 || v > 255) continue; outByte = (uint8_t)v; return true; } return false; } static uint32_t
bytes_to_u32(const uint8_t b[4]) { if (WORD_LITTLE_ENDIAN) { return (uint32_t)b[0] |
((uint32_t)b[1]<<8) | ((uint32_t)b[2]<<16) | ((uint32_t)b[3]<<24); } else { return ((uint32_t)b[0]<<24)
| ((uint32_t)b[1]<<16) | ((uint32_t)b[2]<<8) | (uint32_t)b[3]; } } static int rank_gf2_31(uint32_t
rows[31]) { int rank = 0; for (int col = 30; col >= 0 && rank < 31; --col) { uint32_t mask = (1u << col); int
pivot = -1; for (int r = rank; r < 31; ++r) { if (rows[r] & mask) { pivot = r; break; } } if (pivot < 0)
continue; if (pivot != rank) swap(rows[pivot], rows[rank]); for (int r = 0; r < 31; ++r) { if (r != rank &&
(rows[r] & mask)) rows[r] ^= rows[rank]; } ++rank; } return rank; } int main(int argc, char** argv) {
string filename = "output.dat"; if (argc >= 2) filename = argv[1]; ifstream fin(filename); if (!fin) { cerr
<< "Cannot open " << filename << "\n"; return 1; } const int M_target = 40000; long long N31=0,
N30=0, N29=0, NLE28=0; for (int m=0; m<M_target; ++m) { uint32_t rows[31]; // Each row from one
32-bit word, keep 31 LSBs for (int r=0; r<31; ++r) { uint8_t b[4]; for (int k=0; k<4; ++k) { if
(!read_hex_byte(fin, b[k])) { cerr << "EOF before completing matrix " << (m+1) << "\n"; goto done; } }
uint32_t w = bytes_to_u32(b); rows[r] = (w & 0x7FFFFFFFu); // 31 LSBs } int rk = rank_gf2_31(rows);
if (rk==31) ++N31; else if (rk==30) ++N30; else if (rk==29) ++N29; else ++NLE28; } done: long long M =
N31 + N30 + N29 + NLE28; if (M==0) { cerr << "No matrices processed.\n"; return 1; } double E31 =
M*P31, E30 = M*P30, E29 = M*P29, ELE28 = M*PLE28; double chi2 = ((N31-E31)*(N31-E31))/E31 +
((N30-E30)*(N30-E30))/E30 + ((N29-E29)*(N29-E29))/E29 + ((NLE28-ELE28)*(NLE28-ELE28))/ELE28;
// df=3 p-value (closed form): p = erfc(sqrt(x)) + (2/sqrt(pi))*sqrt(x)*exp(-x), x=chi2/2 double x =
0.5*chi2; double sx = sqrt(x); double p = erfc(sx) + (2.0/sqrt(M_PI))*sx*exp(-x); cout << "Binary Rank
31x31 (Diehard)\n"; cout << "M=" << M << "\n"; cout << "N31=" << N31 << " N30=" << N30 << "
N29=" << N29 << " N<=28=" << NLE28 << "\n"; cout << "E31=" << E31 << " E30=" << E30 << " E29="
<< E29 << " E<=28=" << ELE28 << "\n"; cout << "chi2(df=3)=" << chi2 << " p=" << p << "\n"; return 0; }

```

F. What you should observe (rough sanity)

For $M = 40000$, if RNG is good, your observed counts will be near the expected values, e.g.:

32×32 expected (approx)

- Rank32 ~ **11552**
- Rank31 ~ **23103**
- ≤30 ~ **5345**

31×31 expected (approx)

- Rank31 ~ **11552**
- Rank30 ~ **23103**
- Rank29 ~ **5134**
- ≤28 ~ **211**

If you paste your program outputs (the observed counts), I can compute/verify chi-square and tell you if the p-values are reasonable.

You said:

now Rank 6×8 explain it step to step test with full explanation like i dont know anything

ChatGPT said:

Diehard Binary Rank 6×8 test — step by step (from zero)

This test is the “small-matrix” version of the rank tests. It looks for **linear patterns** in your bitstream, but using many tiny matrices so it’s very sensitive.

1) What the test is doing (idea)

1. Take bits from your RNG.
2. Fill a **6×8** matrix (6 rows, 8 columns) with those bits.
3. Compute its **rank over GF(2)** (binary arithmetic).
4. Repeat a large number of times and count how often the rank is 6, 5, 4, ...
5. Compare your observed counts to the **theoretical distribution** for truly random bits.

If your RNG has linear structure, you’ll see **too many low-rank matrices**.

2) GF(2) arithmetic (super important)

All calculations are in GF(2):

- $1 + 1 = 0$ (XOR)
- $1 - 1 = 0$ (also XOR)
- $1 + 0 = 1$
- Multiply is AND.

So Gaussian elimination is done using **XOR row operations** only.

3) How one 6×8 matrix is formed from the stream

A 6×8 matrix has:

$$6 \times 8 = 48 \text{ bits}$$

That is exactly **6 bytes**.

Step-by-step fill rule

For each matrix:

1. Read **6 bytes** from the stream.
2. Each byte becomes one row (8 bits).

So:

- Row 1 = bits of byte1
- Row 2 = bits of byte2
- ...
- Row 6 = bits of byte6

Example: if the next 6 bytes are:

12 AF D3 56 00 9C

then row1 is bits of 0x12, row2 is bits of 0xAF, etc.

Bit order inside a byte (MSB→LSB or LSB→MSB) does **not** change the rank (it only permutes columns). Pick one and be consistent.

4) How to compute rank of a 6×8 binary matrix (step-by-step)

You treat each row as an 8-bit value (0–255).

Let rows be `r[0..5]` each stored in `uint8_t`.

Elimination procedure (same as 32×32, just smaller)

Initialize rank = 0.

For each column from bit 7 down to bit 0:

1. **Find pivot:** find a row $i \geq \text{rank}$ such that it has a 1 in this column.
2. If found:
 - swap that row with row rank
 - XOR it into every other row that has a 1 in that column (to clear that column)
 - increase rank++
3. If no pivot exists in that column, move to next column.

Final rank is between 0 and 6.

5) What ranks are possible and what you count

Because there are only 6 rows, the maximum rank is 6.

You count how many matrices have:

- rank 6
- rank 5
- rank 4
- rank 3
- rank 2

- rank 1
- rank 0

In practice, ranks 0–2 are extremely rare for random data, but you still count them.

6) Expected probabilities (what “random” should produce)

For a random **6×8** matrix over GF(2), the probability that the matrix has rank r is known exactly.

A convenient way to compute it is:

Number of $m \times n$ binary matrices of rank r over GF(2):

$$N_{m,n}(r) = \frac{\prod_{i=0}^{r-1} (2^m - 2^i) \prod_{i=0}^{r-1} (2^n - 2^i)}{\prod_{i=0}^{r-1} (2^r - 2^i)}$$

Total number of $m \times n$ binary matrices is 2^{mn} .

So:

$$P(\text{rank} = r) = \frac{N_{m,n}(r)}{2^{mn}}$$

Here $m = 6$, $n = 8$, $mn = 48$, and $r = 0..6$.

✓ This gives the exact expected distribution.

Practical note: Original Diehard hard-codes these probabilities (it prints expected counts for a fixed number of matrices). If you want, I can compute the exact numeric probabilities for all ranks 0–6 and give you the expected counts for your chosen M .

7) How much data is used

Each matrix uses **6 bytes**.

So for M matrices:

$$\text{bytes} = 6M$$

Example:

- $M = 100,000 \Rightarrow 600,000$ bytes (~0.6 MB)
- $M = 1,000,000 \Rightarrow 6,000,000$ bytes (~6 MB)

Diehard typically uses a large M here because the test is cheap.

8) Final step: chi-square and p-value

You compute observed counts N_r for ranks $r = 0..6$.

Compute expected counts:

$$E_r = MP_r$$

Then:

$$\chi^2 = \sum_{r=0}^6 \frac{(N_r - E_r)^2}{E_r}$$

Degrees of freedom:

- 7 categories $\rightarrow df = 7 - 1 = 6$

Compute p-value from χ^2 with $df=6$.

9) Summary in one line

Read 6 bytes $\rightarrow$ form 6 $\times$ 8 bit matrix $\rightarrow$ compute GF(2) rank $\rightarrow$ repeat many times $\rightarrow$ compare rank histogram to theoretical random histogram with chi-square p-value.

If you tell me how many matrices you want to use (e.g., **100,000** or **1,000,000**) and confirm your input is the same **hex-byte text format**, I'll give you:

1. the **exact numerical expected probabilities for ranks 0..6**, and
2. a **fast C++ implementation** matching your output.dat style.

You said:

what is expected count of rank here

ChatGPT said:

Thought for 11s

In the **original Diehard Rank 6 $\times$ 8** test, Diehard **pools ranks into 3 buckets** (because very small ranks are rare):

- **Rank = 6**
- **Rank = 5**
- **Rank ≤ 4** (i.e., ranks 0,1,2,3,4 all combined)

For a truly random stream, the **theoretical probabilities** are approximately:

- $P(r = 6) \approx 0.773118$
- $P(r = 5) \approx 0.217439$

- $P(r \leq 4) \approx 0.009443$

Expected counts

If you process **M matrices**, expected counts are:

- $E_6 = 0.773118 M$
- $E_5 = 0.217439 M$
- $E_{\leq 4} = 0.009443 M$

In classic Diehard printouts, they often use **M = 100,000 matrices** (per chosen byte/bit-window), giving:

- **Expected rank ≤ 4 :** $E_{\leq 4} \approx 944.3$
- **Expected rank =5:** $E_5 \approx 21743.9$
- **Expected rank =6:** $E_6 \approx 77311.8$

If you tell me what **M** you are using in your own suite (e.g., 100000 or 40000), I'll compute the exact expected numbers for your run size.

You said:

how much data it needed for this tes

ChatGPT said:

For the **Diehard Binary Rank 6×8** test:

Data needed per matrix

A 6×8 binary matrix has $6 \times 8 = 48$ bits = **6 bytes**.

So the test consumes:

$$\text{data} = 6M \text{ bytes}$$

where M = number of 6×8 matrices you test.

In the original Diehard settings

Diehard typically uses **M = 100,000 matrices** for the 6×8 rank test.

So required data is:

$$6 \times 100000 = 600000 \text{ bytes} \approx 0.6 \text{ MB}$$

✓ **Original Diehard: ~600,000 bytes** of random data for Rank 6×8.

Quick table

Matrices (M)	Bytes needed	Approx
10,000	60,000	60 KB
40,000	240,000	234 KB
100,000	600,000	0.6 MB
1,000,000	6,000,000	6 MB

If you want, I can also give you the **C++ code** for this test in the same style as your 32×32 and 31×31 (reading hex bytes from output.dat, counting ranks into Diehard’s 3 buckets: 6, 5, ≤4, then chi-square + p-value).

You said:

tyyes

ChatGPT said:

👍 Great. If you want to finish the **Rank 6×8** test next, here’s the exact checklist (same style as your other two):

- Use **M = 100,000 matrices** (classic Diehard setting)
- Data needed: **600,000 bytes** (i.e., 6 bytes per matrix)
- Buckets: **Rank 6, Rank 5, Rank ≤ 4**
- Expected (for M matrices):
 - $E_6 = 0.773118 M$
 - $E_5 = 0.217439 M$
 - $E_{\leq 4} = 0.009443 M$

Whenever you say “give code for 6×8”, I’ll write the .cpp in the same output.dat hex-token format with chi-square + p-value output.

You said:

yes

ChatGPT said:

```
// diehard_rank6x8.cpp // // Diehard Binary Rank 6x8 test (original-style bucketing): // Bucket A:
rank = 6 // Bucket B: rank = 5 // Bucket C: rank <= 4 // // INPUT : output.dat (TEXT file containing HEX
BYTES, e.g. "0A FF 1c ...") // DEFAULT: M = 100000 matrices (classic Diehard choice) // DATA : 6 bytes
per matrix => total bytes needed = 6*M // // Expected probabilities (random bits): // P(rank=6) ≈
```

```
0.773118 // P(rank=5) ≈ 0.217439 // P(rank≤4) ≈ 0.009443 // // Chi-square with 3 buckets => df = 2
// For df=2, p-value = exp(-chi2/2) // // Compile: // g++ -O3 -std=c++17 diehard_rank6x8.cpp -o
diehard_rank6x8 // // Run (reads output.dat, M=100000): // ./diehard_rank6x8 // // Run with
custom file and/or M: // ./diehard_rank6x8 output.dat 200000 #include <cstdint> #include <cstdlib>
#include <cmath> #include <iostream> #include <fstream> #include <string> using namespace std; //
Expected probabilities for rank buckets (6x8 over GF(2)) static constexpr double P6 = 0.773118; static
constexpr dou
```

ChatGPT can make mistakes. Check important info. See Cookie Preferences.